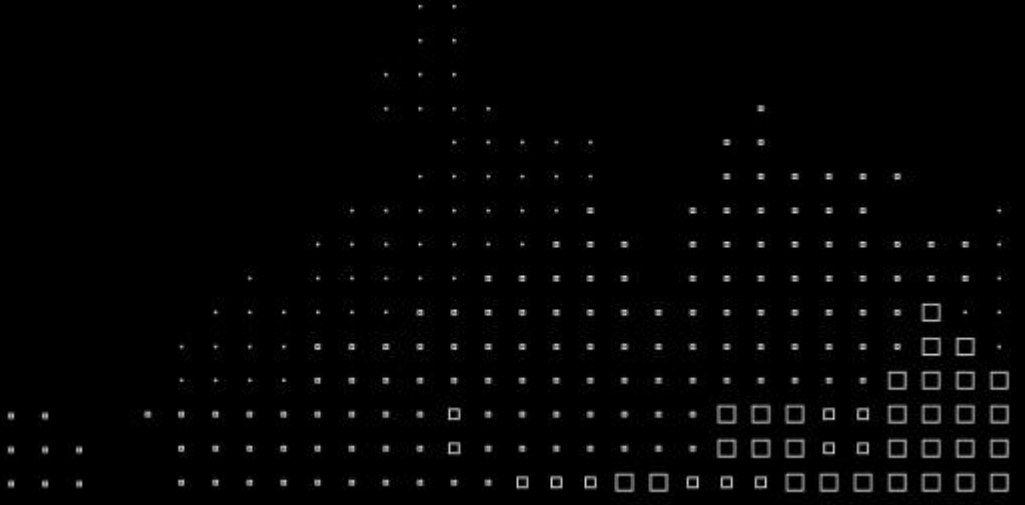
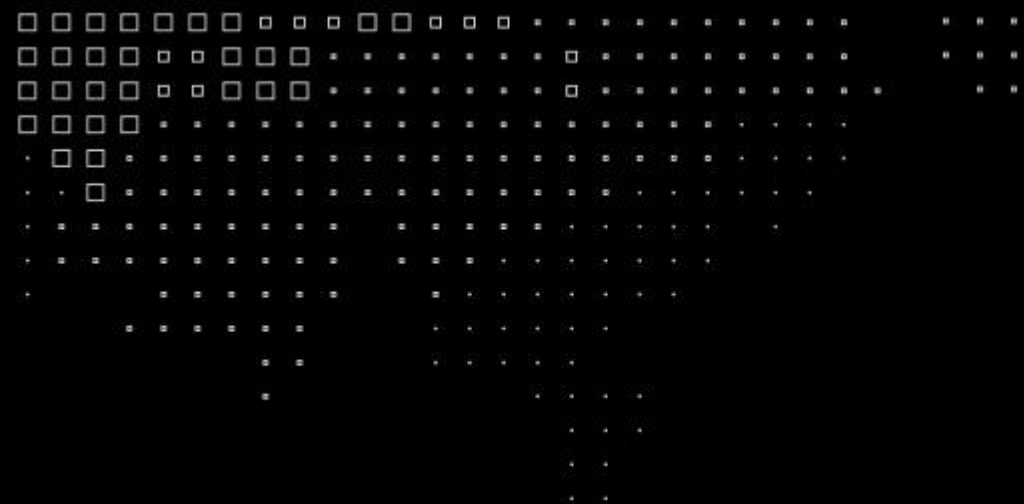


MBA⁺

**AI Engineering &
Multi-Agents**

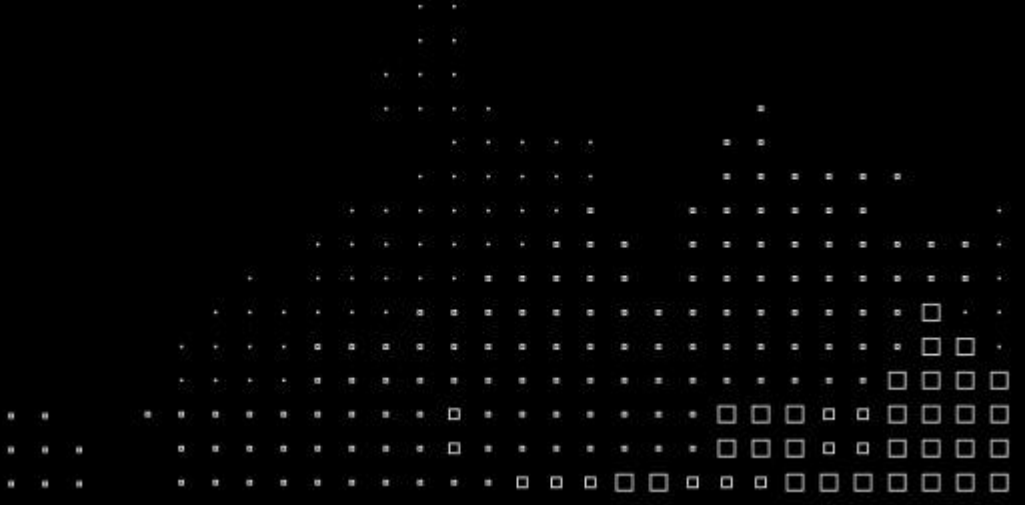




MBA⁺

**Data Architecture,
Integration and Ingestion**

Prof.: Leandro Mendes



Data Architecture, Integration and Ingestion

(O que vamos explorar?)

Aula 1 – 11/ago (ter)

- Sistemas de Gerenciamento de banco de dados
- Estratégias de arquitetura
- Data Warehouse, Data lake, Lakehouse e Data Mesh

Aula 2 – 13/ago (qui)

- Exemplos de Bancos, diferenças e usos:
 - Bancos Relacionais
 - Bancos Colunares

Aula 3 – 20/ago (qui)

- Exemplos de Bancos, diferenças e usos:
 - Bancos de documentos
 - Bancos de Grafos & Vector

Aula 4 – 27/ago (qui)

- Ingestão de dados, tratamentos e manipulações
- Pipeline de dados, governança e qualidade
- Integração de dados
 - Cargas batch, ETL, vantagens e desvantagens

Aula 5 – 03/set (qui)

- Eventos, APIs e casos de uso
- Visualização de dados
- Boas práticas e recomendações

Bancos de documentos

Características:

- ✓ Emparelha cada chave com uma estrutura de dados complexa conhecida como um documento. Os documentos podem conter muitos pares chave-valor diferentes, ou pares de matriz de chaves, ou mesmo documentos aninhados.
- ✓ Os documentos são armazenados no formato JSON
- ✓ Foco no armazenamento do dado e não no relacionamento

Usos:

- ✓ Aplicações de distribuição de conteúdo
- ✓ Aplicações que trabalham com JSON
- ✓ Estruturas variáveis
- ✓ Perfis de usuário

DB-Engines Ranking of Document Stores

The DB-Engines Ranking ranks database management systems according to their popularity. The ranking is updated monthly.

This is a partial list of the [complete ranking](#) showing only document stores.

Read more about the [method](#) of calculating the scores.



☐ include secondary database models

58 systems in ranking, April 2024

Rank			DBMS	Database Model	Score		
Apr 2024	Mar 2024	Apr 2023			Apr 2024	Mar 2024	Apr 2023
1.	1.	1.	MongoDB +	Document, Multi-model ⓘ	423.96	-0.57	-17.93
2.	2.	2.	Amazon DynamoDB +	Multi-model ⓘ	77.57	-0.15	+0.12
3.	3.	3.	Databricks +	Multi-model ⓘ	76.33	+1.99	+15.36
4.	4.	4.	Microsoft Azure Cosmos DB +	Multi-model ⓘ	29.85	-0.54	-5.23
5.	5.	5.	Couchbase +	Document, Multi-model ⓘ	18.46	-0.69	-5.30
6.	6.	6.	Firebase Realtime Database	Document	15.00	-0.07	-3.22
7.	7.	7.	CouchDB	Document, Multi-model ⓘ	10.26	-1.47	-4.36
8.	8.	8.	Google Cloud Firestore	Document	8.96	-1.01	-2.13
9.	9.	↑ 10.	Realm	Document	7.71	+0.01	-0.57
10.	10.	↓ 9.	MarkLogic	Multi-model ⓘ	6.50	-0.57	-1.84
11.	11.	11.	Aerospike +	Multi-model ⓘ	6.10	-0.41	-0.30
12.	12.	↑ 13.	Google Cloud Datastore	Document	4.49	-0.39	-1.41
13.	↑ 15.	↑ 21.	InterSystems IRIS +	Multi-model ⓘ	4.39	+0.57	+1.19

<https://db-engines.com/en/ranking/document+store>

JSON

JSON (JavaScript Object Notation) é um modelo para armazenamento e transmissão de informações no formato texto. Apesar de muito simples, tem sido bastante utilizado por aplicações Web devido a sua capacidade de estruturar informações de uma forma bem mais compacta do que a conseguida pelo modelo XML, tornando mais rápido o parsing dessas informações.

Não tem limitação de tamanho, armazena conjuntos de chave/valor e é suportado pela maioria das linguagens de programação modernas

Ref.: <https://docs.fileformat.com/web/json/>

```
{
  "name": "Jack",
  "age": 30,
  "contactNumbers": [
    {
      "type": "Home",
      "number": "123 123-123"
    },
    {
      "type": "Office",
      "number": "321 321-321"
    }
  ],
  "spouse": null,
  "favoriteSports": [
    "Football",
    "Cricket"
  ]
}
```

Particionamento de dados

MongoDB

O que é?

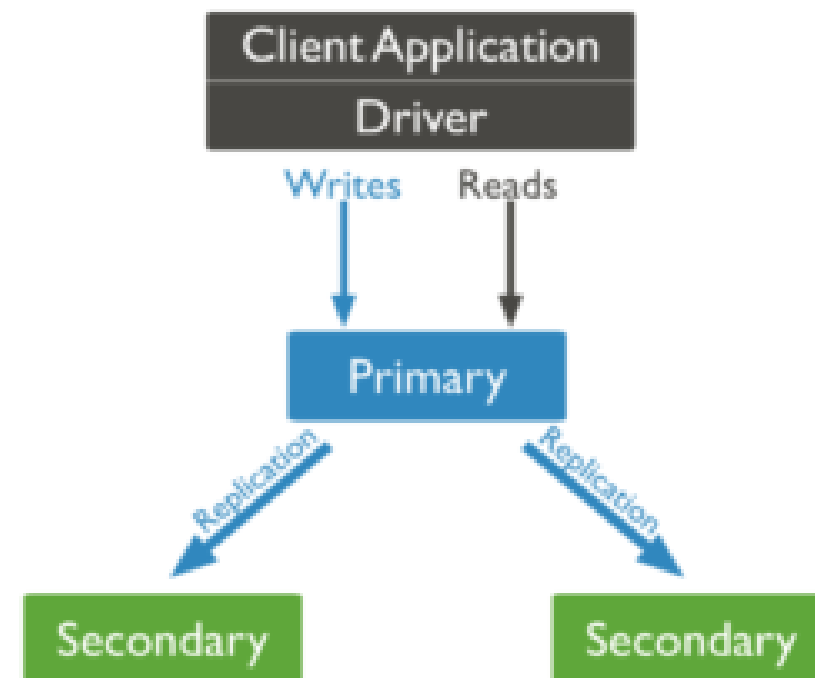
O particionamento de dados (sharding) serve para escalar consultas de dados. As gravações nas coleções de dados são feitas nas partições primárias e replicadas para as partições secundárias.

Benefícios

Essa abordagem favorece aplicações que tenham mais operações de consultas do que gravação. Uma vez que o dado sempre é gravado através da partição primária, mas pode ser consultado através de todas as partições que possuam o dado.

Limitações

Este modelo deixa explícita sua limitação na gravação dos dados. Portanto, as leituras podem ser escaláveis e flexíveis, já as operações de gravação são limitadas



Em caso de falha em um nó primário, um dos secundários é promovido a primário

Vantagens dos bancos de documentos

Armazenamento: diferentes de bancos relacionais, um campo que não é informado para um registro não ocupa armazenamento.

Flexibilidade de esquema: os documentos json armazenados podem conter qualquer conteúdo dentro de uma mesma coleção de dados

Agrupamento de dados: como não é relacional, os dados de uma mesma entidade são guardados juntos (ex: O documento do cliente pode conter todos os dados do cliente)

Chaves estrangeiras: Como não há relacionamento, não há necessidade de armazenar as chaves estrangeiras em várias tabelas

Quando usar um banco de documento

Casos de uso: aqui estão alguns casos de uso populares dos bancos de dados de document-based:

- ✓ Catálogo de produtos de comércio eletrônico;
- ✓ Blogs e gerenciamento de conteúdo;
- ✓ Gerenciamento de configurações;
- ✓ Manutenção de dados baseados em localização - dados geoespaciais;
- ✓ Sites de redes móveis e sociais;
- ✓ Objetivos fracamente acoplados - o design pode mudar ao longo do tempo;

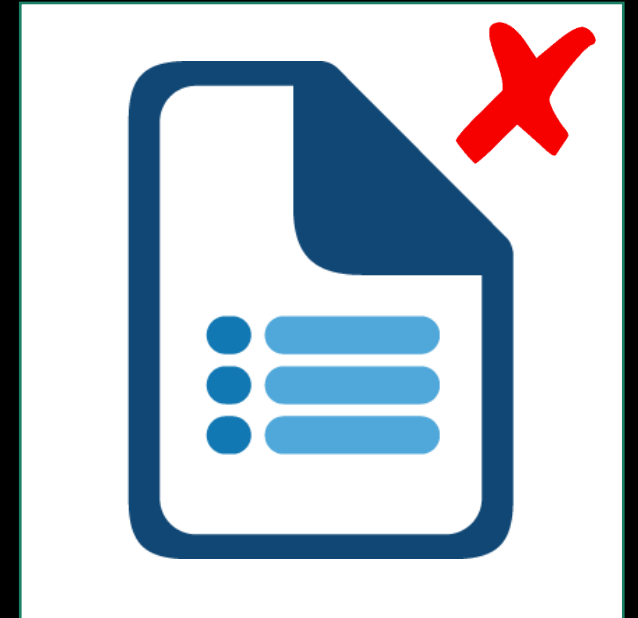


45697056

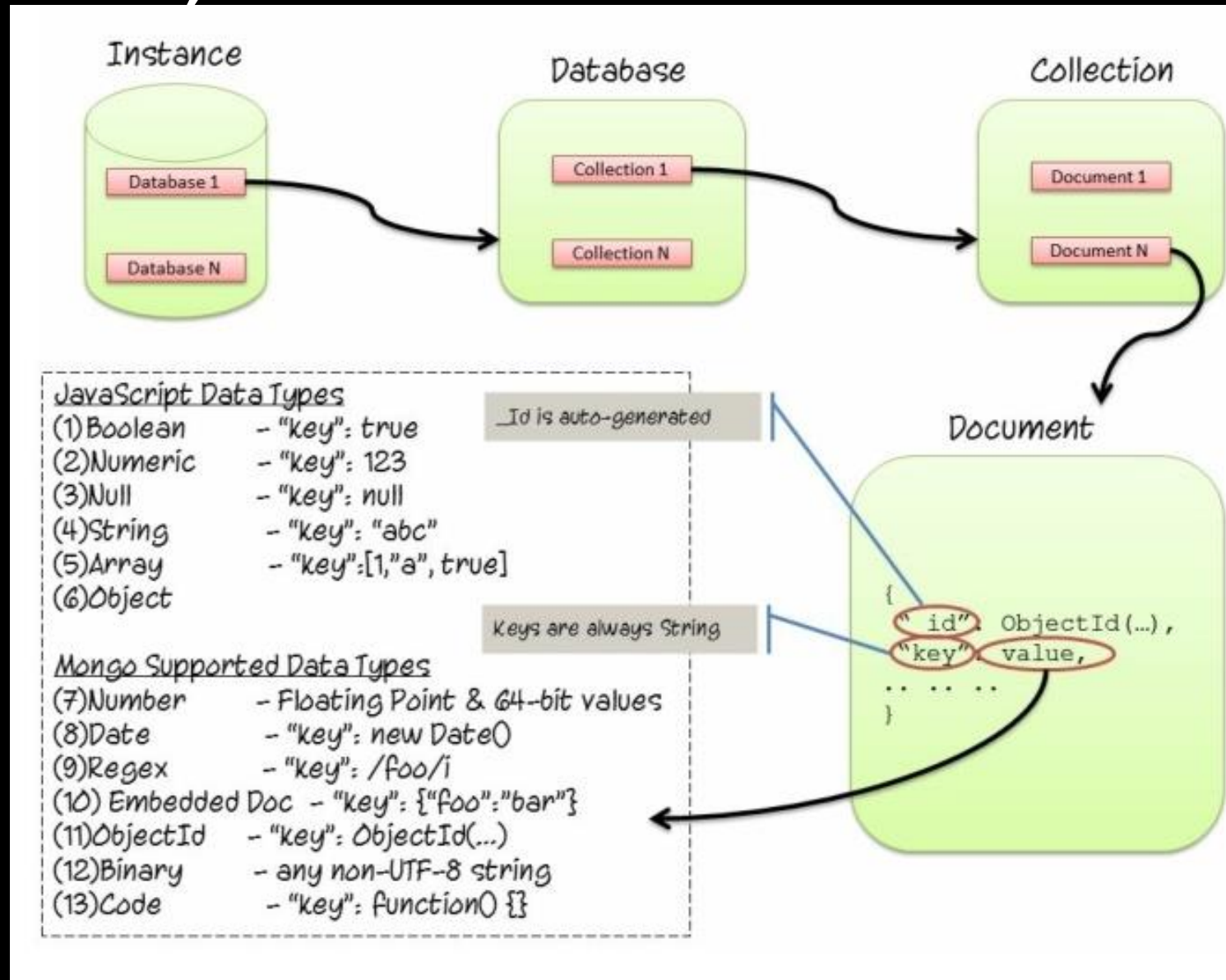
Quando **não** usar um banco de documento

No entanto, os bancos de dados document-based não são a escolha ideal para cada caso de uso quando:

- ❖ Sistemas altamente transacionais ou em que o modelo de dados é projetado antecipadamente;
- ❖ Sistemas fortemente acoplados;
- ❖ Não são a escolha certa se você precisar executar consultas de pesquisa complexas ou se o seu aplicativo exigir transações complexas com várias operações.



Database, Collections e Documents



Modelo de Dados

Relational

Person:

Pers_ID	Surname	First_Name	City
0	Miller	Paul	London
1	Ortega	Alvaro	Valencia
2	Huber	Urs	Zurich
3	Blanc	Gaston	Paris
4	Bertolini	Fabrizio	Rom

Car:

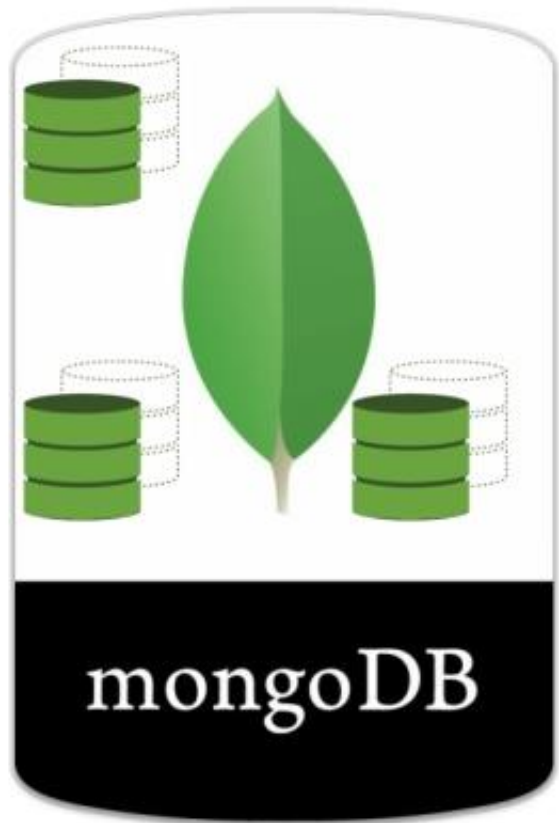
Car_ID	Model	Year	Value	Pers_ID
101	Bentley	1973	100000	0
102	Rolls Royce	1965	330000	0
103	Peugeot	1993	500	3
104	Ferrari	2005	150000	4
105	Renault	1998	2000	3
106	Renault	2001	7000	3
107	Smart	1999	2000	2

no relation

MongoDB Document

```
{
  first_name: 'Paul',
  surname: 'Miller'
  city: 'London',
  location: [45.123,47.232],
  cars: [
    { model: 'Bentley',
      year: 1973,
      value: 100000, ... },
    { model: 'Rolls Royce',
      year: 1965,
      value: 330000, ... }
  ]
}
```

Databases



Criando Databases:

```
USE <database name>
```

```
USE analyses
```

```
USE mydb
```

Listar os Databases:

```
SHOW dbs
```

Apagando um Database:

```
db.dropDatabase()
```

Database – Cont.

Outras funções de Database:

```
db.cloneCollection()  
db.cloneDatabase()  
db.createView()  
db.getCollectionNames()  
db.runCommand()
```

Collections – Cont.

Exibindo as Coleções:

```
show collections
```

Criação automática de Coleções:

```
db.faculdades.insert({"name" : "fiap"})  
show collections  
mycol  
restaurantes  
system.indexes  
faculdades  
...
```

Collections – Cont.

- Removendo as Coleções:

```
db.collection.drop()
```

Exemplo de remoção de Coleções:

```
use mydb
switched to db mydb

show collections
mycol
restaurantes
system.indexes
faculdades

db.faculdades.drop()
```

Collections – Cont.

Outras funções de Coleções:

```
db.collection.count()
db.collection.dataSize()
db.collection.deleteOne()
db.collection.deleteMany()
db.collection.distinct()
db.collection.explain()
```


Datatypes



Primitive Data Types	boolean
	byte
	short
	char
	int
	long
	float
	double
Wrapper Types	Boolean
	Byte
	Short
	Character
	Integer
	Long
	Float
	Double
Date/ Time Types	java.util.Date (TemporalType.DATE)
	java.util.Date (TemporalType.TIME)
	java.util.Date (TemporalType.TIMESTAMP)
	java.sql.Date
	java.sql.Time
	java.sql.Timestamp
	java.util.Calendar
Other Types	String
	Object
	java.math.BigInteger
	java.math.BigDecimal
	java.util.UUID

Documents

```
{
  _id: ObjectId(7df78ad8902c)
  title: 'MongoDB Overview',
  description: 'MongoDB is no sql database',
  by: 'tutorials point',
  url: 'http://www.tutorialspoint.com',
  tags: ['mongodb', 'database', 'NoSQL'],
  likes: 100,
  comments: [
    {
      user: 'user1',
      message: 'My first comment',
      dateCreated: new Date(2011,1,20,2,15),
      like: 0
    },
    {
      user: 'user2',
      message: 'My second comments',
      dateCreated: new Date(2011,1,25,7,45),
      like: 5
    }
  ]
}
```

Documents

Criando Documentos:

```
db.collection.insert(document)
```

Exemplo:

```
db.mycol.insert({  
  _id: ObjectId(7df78ad8902c),  
  title: 'MongoDB Overview',  
  description: 'MongoDB is no sql database',  
  by: 'tutorials point',  
  url: 'http://www.tutorialspoint.com',  
  tags: ['mongodb', 'database', 'NoSQL'],  
  likes: 100  
})
```

Exercício:

Explorando um banco Mongodb



<https://www.mongodb.com/>

Continuação exercício MongoDB...

FIAP



- Continuação do Laboratório SQL / Cassandra da aula anterior.

Você já tem `invent\_mestre`, `transac\_mestre` e a tabela do seu departamento.
Agora o mesmo sistema vai para um banco de documentos. Use IA para gerar os scripts — seu papel é avaliar e validar o que ela produzir.

Bancos de dados de grafos

Funcionamento de bancos de grafos

Um banco de dados gráfico armazena nós e relacionamentos em vez de tabelas ou documentos.

Os dados são armazenados da mesma forma que você esboça ideias em um quadro branco.

Não há restrição a um modelo pré-definido, permitindo uma forma muito flexível de os pensar e utilizar.

DB-Engines Ranking of Graph DBMS

The DB-Engines Ranking ranks database management systems according to their popularity. The ranking is updated monthly.

This is a partial list of the [complete ranking](#) showing only graph DBMS.

Read more about the [method](#) of calculating the scores.



☐ include secondary database models

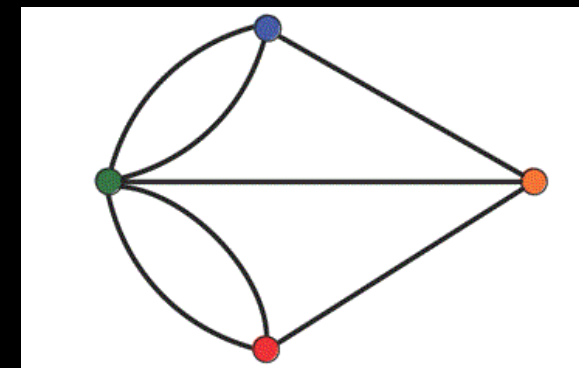
41 systems in ranking, April 2024

Rank			DBMS	Database Model	Score		
Apr 2024	Mar 2024	Apr 2023			Apr 2024	Mar 2024	Apr 2023
1.	1.	1.	Neo4j	Graph	44.47	+0.11	-7.13
2.	2.	2.	Microsoft Azure Cosmos DB	Multi-model	29.85	-0.54	-5.23
3.	3.	3.	Aerospike	Multi-model	6.10	-0.41	-0.30
4.	4.	4.	Virtuoso	Multi-model	4.20	-0.19	-2.04
5.	5.	5.	ArangoDB	Multi-model	3.77	-0.45	-1.03
6.	6.	6.	OrientDB	Multi-model	3.27	-0.11	-0.79
7.	8.	9.	GraphDB	Multi-model	3.10	+0.19	+0.76
8.	7.	11.	Memgraph	Graph	3.00	-0.09	+0.88
9.	9.	7.	Amazon Neptune	Multi-model	2.58	-0.24	-0.11
10.	10.	10.	NebulaGraph	Graph	2.12	-0.24	-0.05
11.	12.	14.	Stardog	Multi-model	2.05	+0.02	+0.34
12.	11.	8.	JanusGraph	Graph	1.91	-0.13	-0.53

Teoria dos grafos

1793

Leonard Euler, através do mito das pontes de Königsberg, comprovou através de grafos que não era possível resolver essa façanha;



A ideia de Grafo surgiu independente das diversas áreas de conhecimento, no entanto é considerada como uma área da matemática aplicada. A mais antiga menção sobre o assunto ocorreu no trabalho de **Euler** (pronuncia-se Óiler), no ano de 1736 para modelar e explicar um problema chamado “**Pontes de Königsberg**” (atual Kaliningrado). Seis das sete pontes interligavam duas ilhas às margens do Rio Pregel e uma fazia a ligação entre as duas ilhas. O problema consistia na seguinte questão: **como seria possível fazer um passeio a pé pela cidade de forma a se passar uma única vez por cada uma das sete pontes e retornar ao ponto de partida?**

Euler focou apenas no problema removendo todos os detalhes geométricos do mesmo (distância, tamanho das pontes, formas etc) e apresentou uma solução que além de provar que não era possível passar apenas uma vez por cada ponte, como a forma de solução poderia ser aplicada a outros problemas semelhantes. Assim surgiu a teoria dos grafos.

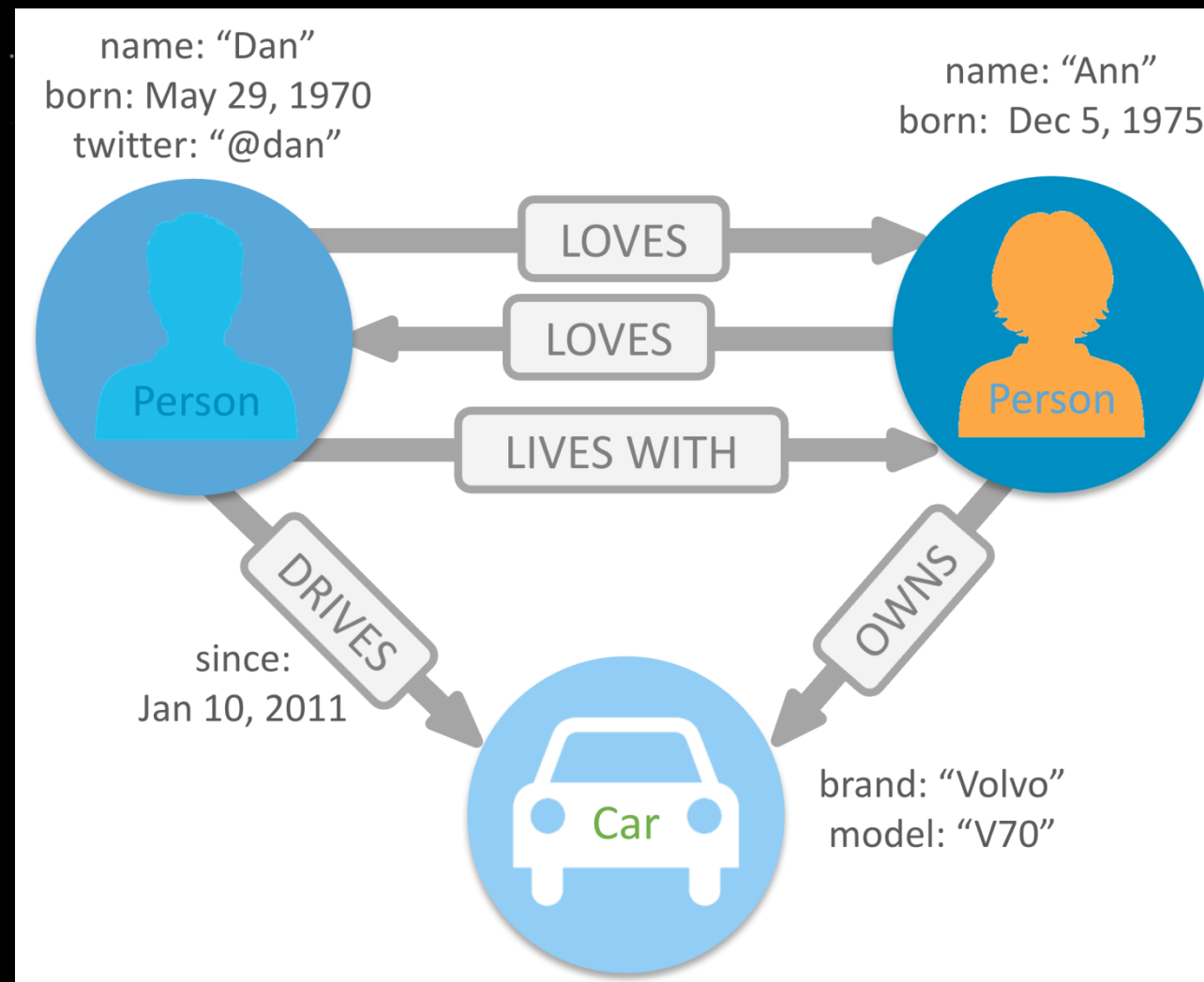


Banco de grafos: Neo4j

+

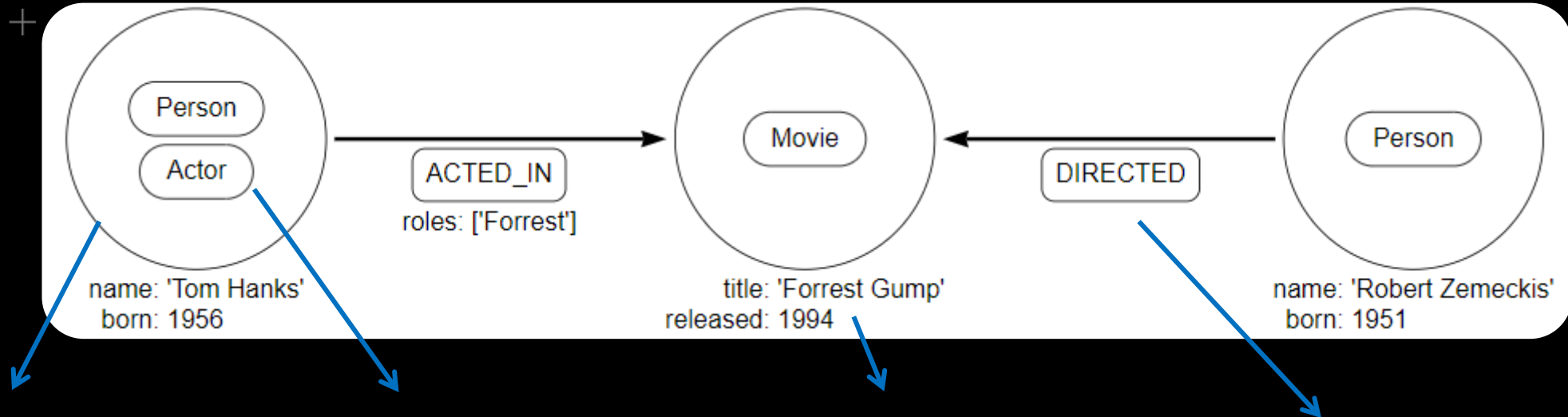
O **Neo4j** é um banco de dados Open Source baseado no conceito NoSQL (Banco de Dados que não utiliza os conceitos estruturados).

As informações não são armazenadas em tabelas, mas sim na forma de Grafos e suas estruturas são representadas de forma que o conhecimento é representado pelos conceitos matemáticos da Teoria de Grafos.



<https://neo4j.com/developer/graph-database/>

Neo4j



NÓ

- ✓ São os objetos de um domínio
- ✓ Podem ter zero ou mais **rótulos** (labels)
- ✓ Podem ter zero ou mais **propriedades**
- ✓ Podem ter zero ou mais **relacionamentos**

RÓTULO (Label)

- ✓ Definem domínios através do agrupamentos de nós.
- ✓ Podem ser adicionados e removidos livremente

PROPRIEDADES

- ✓ Pares de chave-valor usados para armazenar dados sobre os nós
- ✓ Podem ser numéricos strings ou booleanos

RELACIONAMENTO

- ✓ São conexões do tipo origem-destino entre os nós
- ✓ Podem ligar um nó a ele próprio
- ✓ Podem possuir propriedades

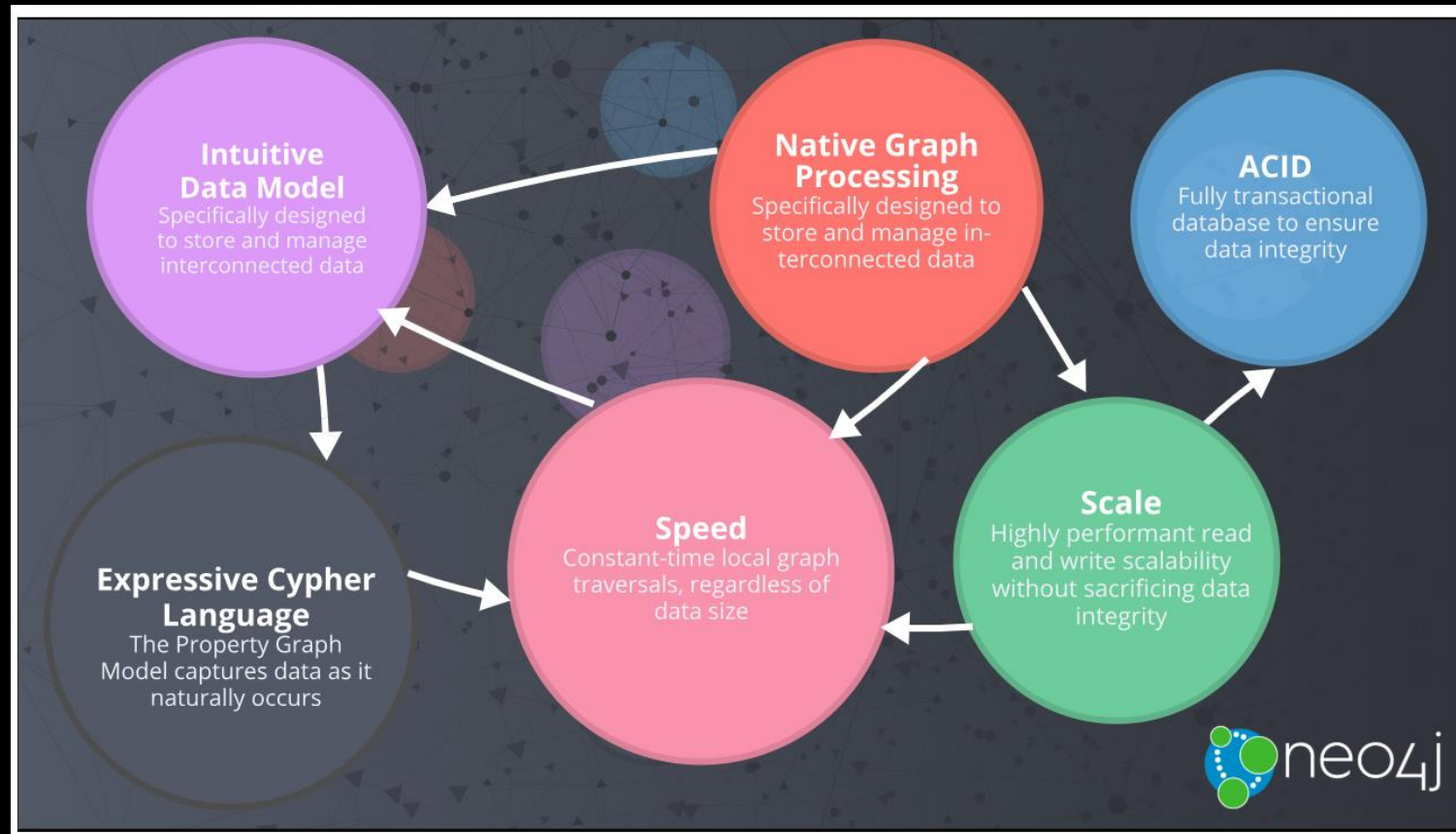
Características do Neo4j

- ✓ **Dados não estruturados;**
- ✓ **Usa o armazenamento gráfico nativo com o GPE nativo (Graph Processing Engine);**
- ✓ **JSON e XLS support;**
Suporta a exportação de dados de consulta para o formato JSON e XLS;
- ✓ **Ele fornece a API REST para ser acessada por qualquer linguagem de programação, como Java, Spring, Scala etc.**
- ✓ **Schema-less e Type-less;**
- ✓ **Suporte Full a ACID;**
A integridade dos dados por meio de transações compatíveis com ACID.

Características do Neo4j

- ✓ **Free-for-all**
Não adiciona constraints;
- ✓ **Suporta 34.4 BI de nodes e 34.4BI de relationships;**
- ✓ **Não faz sharding de subgrafos, limitando o tamanho dos grafos;**
- ✓ **Opções de Cluster HA com o uso da versão Enterprise;**
Gerenciamento de memória fora do heap, Causal Clustering, que otimiza tanto o acesso somente leitura quanto o acesso de leitura / gravação, alta disponibilidade (HA), recuperação de desastre e suporte a vários data centers.
- ✓ **Cypher - Linguagem (Query) para Neo4j;**
É uma linguagem de consulta legível por humanos projetada especificamente para manipular dados gráficos conectados.

Neo4J: Características

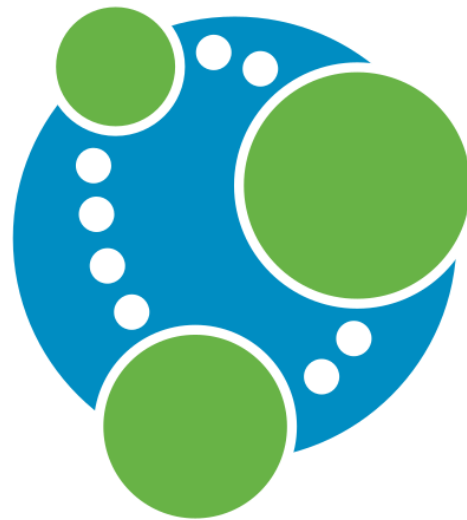


Aplicações de bancos de grafos

- ✓ Sistemas de recomendações;
- ✓ Usado para Business Intelligence;
- ✓ Sistema Geoespaciais;
- ✓ O mundo é baseado em relacionamentos;
- ✓ Os dados e suas estruturas são dinâmicos

Exercício:

Explorando um banco neo4j



neo4j

Ref.: <https://neo4j.com/docs/cypher-manual/current/introduction/>

Exercício Neo4j



O Docker já está instalado e uma imagem chamada *neo4j* já está disponível. Siga os passos iniciais abaixo:

Script disponibilizado com os comandos

Comandos básicos:

1. Criar um container a partir da imagem publicando a porta 7474
2. Listar os comandos do neo4j e neo4j-admin
3. Iniciar o serviço do neo4j
4. Verifique os diretórios configurados
5. Verificar o status do servidor neo4j
6. Acessar o console em localhost:7474 (*)
7. Definir uma nova senha "fiapdtsdaii" (*)

Usuário: neo4j Senha: neo4j

A tela inicial será aberta

Continuação exercício Neo4j...



Interface gráfica

Editor

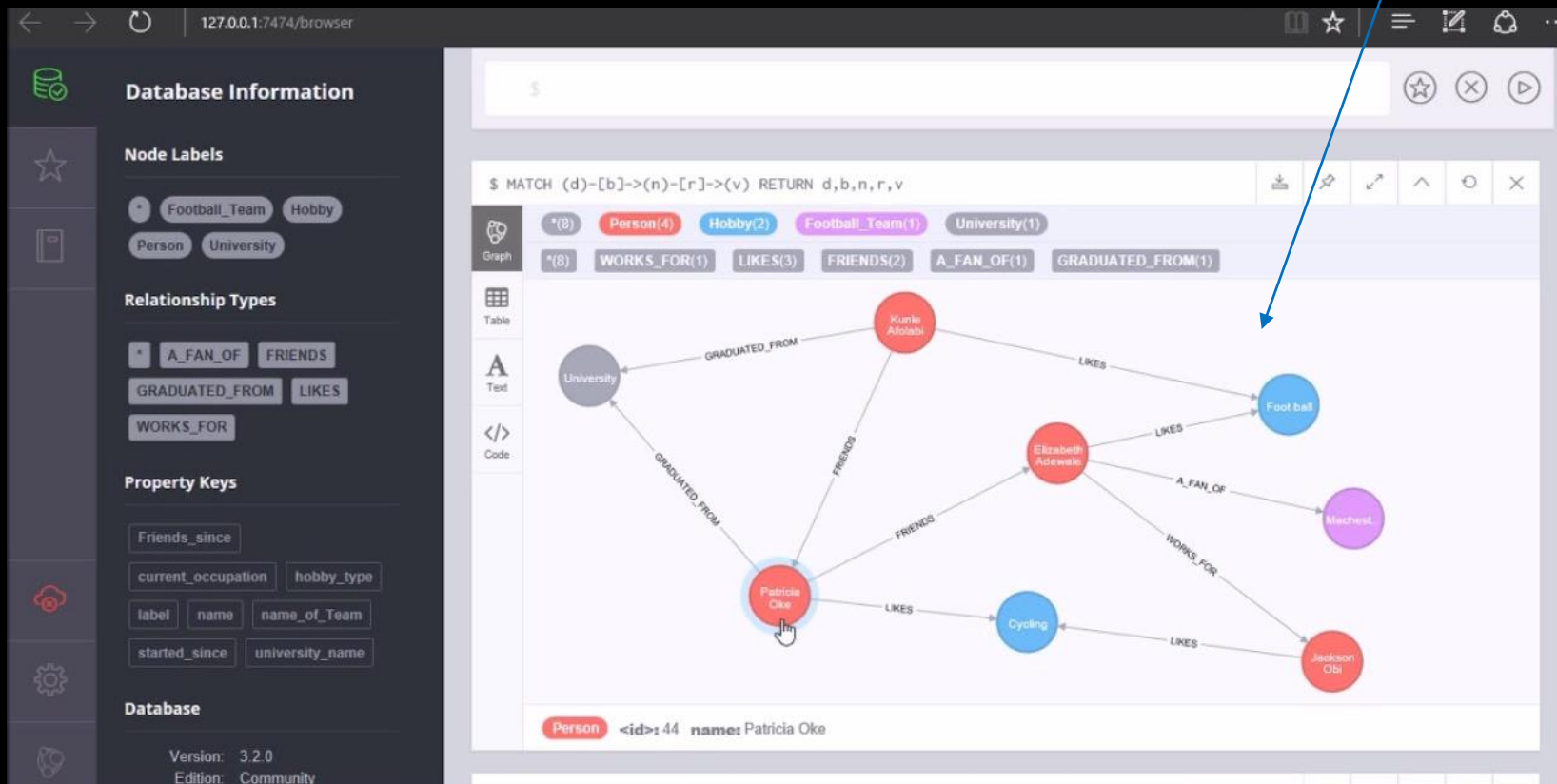
The screenshot shows the Neo4j Browser interface. On the left, there's a sidebar with 'Database Information' (Node Labels: Football_Team, Hobby, Person, University; Relationship Types: A_FAN_OF, FRIENDS, GRADUATED_FROM, LIKES, WORKS_FOR; Property Keys: Friends_since, current_occupation, hobby_type, label, name, name_of_Team, started_since, university_name) and 'Database' (Version: 3.2.0, Edition: Community). The main area displays a graph with nodes like 'University', 'Kunle Adesola', 'Elizabeth Adesola', 'Patricia Oke', 'Football', 'Cycling', and 'Jackson Oke'. Relationships include 'GRADUATED_FROM', 'FRIENDS', 'LIKES', 'A_FAN_OF', and 'WORKS_FOR'. A Cypher query is entered in the top bar: `$ MATCH (d)-[b]->(n)-[r]->(v) RETURN d,b,n,r,v`. Below the query, there are filters for node types and relationship types. The graph is visualized with colored nodes and labeled edges. A tooltip for 'Patricia Oke' shows her ID as 44 and name as Patricia Oke.

O editor é a interface principal para entrar e executar comandos. Insira as consultas do Cypher para trabalhar com dados do gráfico. Use comandos do lado do cliente como: ajuda para outras operações.

- ✓ Edição de linha única para breves consultas ou comandos
- ✓ Mudar para edição multi-linha com <shift-enter>
- ✓ Execute uma consulta com <ctrl-enter>
- ✓ O histórico é mantido para recuperar facilmente os comandos anteriores

Continuação exercício Neo4j...

Interface gráfica



Um quadro de resultado é criado para cada execução de comando, adicionado à parte superior do fluxo para criar uma coleção rolável em ordem cronológica inversa.

- ✓ Quadros especiais como visualização de dados
- ✓ Expandir um quadro para tela cheia
- ✓ Remover um quadro específico do fluxo
- ✓ Limpe o fluxo com o comando **:clear**

Continuação exercício Neo4j...



Interface gráfica

The screenshot shows the Neo4j Browser interface. On the left is a sidebar with 'Database Information', 'Node Labels' (Football_Team, Hobby, Person, University), 'Relationship Types' (A_FAN_OF, FRIENDS, GRADUATED_FROM, LIKES, WORKS_FOR), 'Property Keys' (Friends_since, current_occupation, hobby_type, label, name, name_of_Team, started_since, university_name), and 'Database' (Version: 3.2.0, Edition: Community). The main area displays a Cypher query: `$ MATCH (u:Usuário), (m:ModuloSistema) return *`. Below the query, a table shows the query details: Server version (Neo4j/3.4.0), Server address (localhost:7687), Query (MATCH (u:Usuário), (m:ModuloSistema) return *), Summary ({"statement": {"text": "MATCH (u:Usuário), (m:ModuloSistema)"}}, and Response (["keys": [...]. A blue arrow points to the 'Code' tab in the left sidebar of the main area, which is labeled 'Code'.

A guia de código exibe tudo enviado e recebido do servidor Neo4j, incluindo:

- ✓ Solicitar URI, método HTTP e cabeçalhos
- ✓ Código e cabeçalhos de resposta HTTP de resposta
- ✓ Pedido bruto e conteúdo da resposta no formato JSON

Continuação exercício Neo4j...

Interface gráfica

```
$ MATCH (u:Usuário), (m:ModuloSistema) return *
```

Graph

Table

Text

Code

"m"	"u"
{"Nome": "Compras"}	{"Nome": "Usuário 1", "Id": 1}
{"Nome": "Financeiro"}	{"Nome": "Usuário 1", "Id": 1}
{"Nome": "Compras"}	{"Nome": "Usuário 2", "Id": 2}
{"Nome": "Financeiro"}	{"Nome": "Usuário 2", "Id": 2}

```
$ MATCH (u:Usuário), (m:ModuloSistema) return *
```

Graph

Table

Text

Code

```
{  
  "Nome": "Compras"  
}
```

```
{  
  "Nome": "Usuário 1",  
  "Id": 1  
}
```

```
{  
  "Nome": "Financeiro"  
}
```

```
{  
  "Nome": "Usuário 1",  
  "Id": 1  
}
```

```
{  
  "Nome": "Compras"  
}
```

```
{  
  "Nome": "Usuário 2",  
  "Id": 2  
}
```

Started streaming 4 records after 3 ms and completed after 8 ms.

Continuação exercício Neo4j...



Interface gráfica

The screenshot shows the Neo4j Browser interface. On the left is a sidebar with 'Database Information' including Node Labels (Football_Team, Hobby, Person, University), Relationship Types (A_FAN_OF, FRIENDS, GRADUATED_FROM, LIKES, WORKS_FOR), Property Keys (Friends_since, current_occupation, hobby_type, label, name, name_of_Team, started_since, university_name), and Database details (Version: 3.2.0, Edition: Community). The main area displays a Cypher query: `$ MATCH (d)-[b]->(n)-[r]->(v) RETURN d,b,n,r,v`. Below the query, a graph visualization shows nodes like University, Patricia Oke, Elizabeth Adewale, and Jackson Obi connected by relationships like GRADUATED_FROM, FRIENDS, LIKES, and WORKS_FOR. A blue arrow points to the sidebar with the label 'Sidebar'.

A barra lateral se expande para revelar diferentes painéis funcionais para consultas e informações comuns.

- ✓ Metadados do banco de dados e informações básicas
- ✓ Scripts salvos organizados em pastas
- ✓ Links de informações para documentos e referência
- ✓ Créditos e informações de licenciamento

Continuação exercício Neo4j...



Comandos administrativos...

8. Listar databases
9. Listar usuários
10. Criar um usuário
11. Contar os nós do database

Desafios

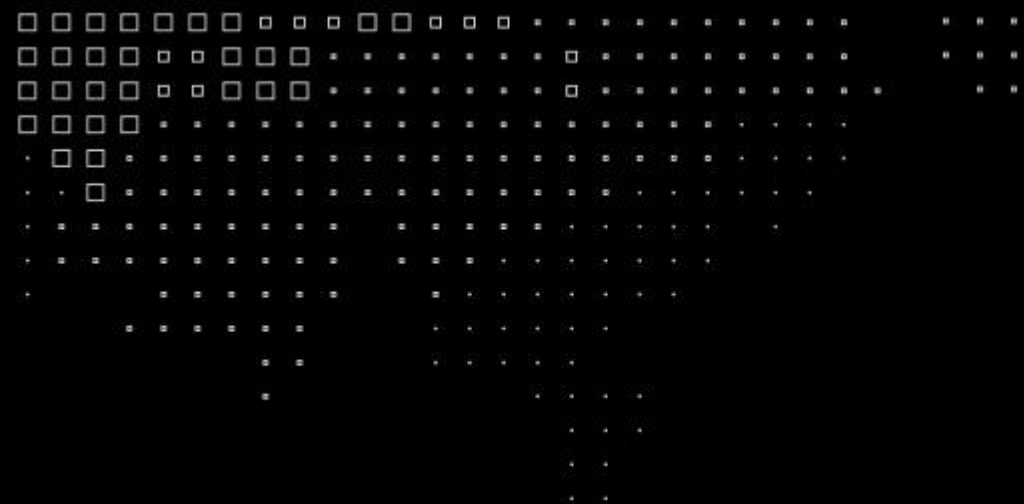


Continuação do Laboratório SQL / Cassandra da aula anterior.

Mesmo sistema de armazém, agora em grafo. Use IA para gerar os scripts — seu papel é avaliar e validar o que ela produzir.

Atividade para atualização de modelagem e interação com dados será compartilhada pelo professor.




MBA⁺

Copyright © 2026 Prof. Leandro Mendes

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor Leandro Mendes

[Material baseado em conteúdo anterior da disciplina \(FIAP\).](#)

